Problem 1

The Trinity Curriculum Development Committee (TCDC) recently submitted a new curriculum, which was approved by the Arts & Sciences Council. The new curriculum requires a student to take a number of classes r_i from m different categories G_i in order to graduate. Let n be the number of courses.

For example, suppose there are $n = 5$ courses A, B, C, D, E and $m = 2$ categories:

- You must take at least $r_1 = 2$ courses from the subset $G_1 = \{A, B, C\}$.
- You must take at least $r_2 = 2$ courses from the subset $G_2 = \{C, D, E\}$.

Then a student who has only taken courses B, C, D cannot graduate, but a student who has taken either A, B, C, D or B, C, D, E can graduate.

Describe and analyze an algorithm to determine whether a given student can graduate. The input to your algorithm is the list of the m categories (each specifying a subset G_i of the n courses and the number of courses r_i that must be taken from that subset) and the list of courses that the student has taken. Justify the correctness of your reduction and analyze the runtime.

Solution: We reduce the problem to maximum flows, as follows. Let L be the list of courses taken by the student for which we wish to determine if they can graduate. Let $G = (V, E)$ be a flow network with edge capacities $c : E \to \mathbb{Z}^+$, where V consists of source and sink vertices s and t, respectively, plus each class C_i and each category G_j. Thus $|V| = n + m + 2 = O(n + m)$. For each class $C_i \in L$ taken by the student, we add the directed edge $C_i \to G_j$ with capacity 1 for each category G_j that includes C_i, i.e., for all categories in $\left\{G_j \;\middle|\; C_i \in G_j\right\}$. Finally, we add directed edges from s to each course C_i with capacity 1, and directed edges from G_j to t with capacity r_j, the number of courses from G_j required in order to graduate. Then $|E| \le nm + n + m = O(nm)$.

Let $R = \sum_j r_j$ be the sum of all requirements. By construction, the maximum flow in G is bounded by the sum of capacities out of s, which is n, as well as the sum of capacities into t, which is R. We claim there is a flow in G of value R if and only if the student can graduate.

- $\underline{\Rightarrow}$: Suppose there is a flow f in G with $|f| \ge R$. As argued above, the sum of capacities into t is R, so all incoming edges to t are saturated in f. A flow decomposition P of f into paths consists of weight-1 paths since all capacities of edges leaving s are 1. Every path includes (exactly) one edge $C_i \to G_j$ between a course C_i and category G_j, which exists if and only if $C_i \in G_j$ and $C_i \in L$. Thus we assign course C_i to satisfy one of the r_j courses from G_j. Assigning courses in this way indeed results in a way for the student to satisfy all requirements, as each course is assigned exactly once because any course is included in at most one path of the decomposition (as its incoming capacity is 1).
- $\underline{\Leftarrow}$: Suppose the student can graduate, so there is a subset of taken classes $L_j \subset C_j$ with $|L_j| = r_j$ that are used to satisfy the requirements of C_j (and no other category). If there are more than r_j classes used to satisfy the category, we pick L_j as any r_j such classes. For each category G_j, we push a unit of flow on the path $s \to C_i \to G_j \to t$ for each taken course $C_i \in G_j$ being used to satisfy the r_j required courses from C_j. Pushing flow in this way satisfies all capacities of the edges since the edges exist, any class is used to satisfy at most one category requirement, and any category has at most r_j courses to satisfy it.

Constructing G takes $O(V + E) = O(nm)$ time, and computing a max-flow in G takes $O(E|f|)$ time via Ford-Fulkerson. We have that $|f| \le \min\{n, R\}$, so the runtime is $O(ER) = O(nmR)$ if $R \le n$, otherwise it is $O(En) = O(n^2 m)$ which matches the runtime $O(EV) = O(n^2 m)$ of Orlin's algorithm.

A reduction to maximum matching: Instead of explicitly reducing to max-flow, we could create the following undirected bipartite graph $G = (\mathscr{C} \sqcup \mathscr{G}, E)$ where $\mathscr{C}$ is the set of courses, $\mathscr{G}$ is the set $\bigcup_j G_j \times \{1, \ldots, r_j\}$; that is, there are r_j copies of category G_j as vertices in $\mathscr{G}$. E contains an edge between $C_i \in \mathscr{C}$ and vertex $(G_j, x) \in \mathscr{G}$ if and only if the student has taken course C_i and G_j includes the course. Then one can show there is a matching of size at least $R = \sum_j r_j$ if and only if the student can graduate. Constructing G takes $O(nm)$ time and computing a maximum matching takes $O(nmR)$ time.

Problem 2

Suppose we are given an array $A[1..m][1..n]$ of non-negative numbers. We want to *round* A to an integer matrix, by replacing each entry x in A with either $\lfloor x \rfloor$ or $\lceil x \rceil$, without changing the sum of entries in any row or column of A. For example:

$$\begin{bmatrix} 1.2 & 3.4 & 2.4 \\ 3.9 & 4.0 & 2.1 \\ 7.9 & 1.6 & 0.5 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 4 & 2 \\ 4 & 4 & 2 \\ 8 & 1 & 1 \end{bmatrix}$$

a. Describe and analyze an efficient reduction-based algorithm that either rounds A in this fashion, or reports correctly that no such rounding is possible. Justify that your algorithm is correct, and analyze the runtime. *[Hint: There is a correct reduction involving a flow network with one vertex per row and one vertex per column.]*

b. Justify that a matrix can be rounded if and only if the sum of entries of each column is an integer and the sum of entries of each row is an integer. *[Hint: Consider a particular* **fractional** *flow on your graph. Recall properties of flow networks with integer capacities.]*

Solution:

a. We reduce the problem to maximum flows, as follows.

Let B be the $m \times n$ matrix where $B[i,j] = A[i,j] - \lfloor A[i,j] \rfloor$; that is, B contains the fractional part of each element in A. It can be verified that A can be rounded if and only if B can be rounded, as rounding only affects the fractional parts of the elements in A. Indeed, A can be rounded to A' if and only if $A' - (A - B)$ is a rounding of B'.[1]

For each row r_i of B, let $w(r_i) = \sum_{1 \le j \le n} B[i,j]$ be the sum of elements in r_i. Similarly, for each column c_j of B, let $w(c_j) = \sum_{1 \le i \le m} B[i,j]$ be the sum of elements in c_j. Finally, let $W = \sum_i w(r_i) = \sum_j w(c_j)$ be the sum of all elements in B.

First, we handle an obvious case directly: if there exists a column c_j where the sum of its entries is not an integer or a row where the sum of its entries is not an integer, then we report IMPOSSIBLE since any rounding of B yields integral row and column sums. Otherwise, we proceed by constructing a flow network.[2]

Let $G = (V, E)$ be a flow network with edge capacities $c : E \rightarrow \mathbb{Z}^+$, where V consists of source and sink vertices s and t, respectively, plus a vertex r_i for each of the m rows of B and a vertex c_j for each of the n columns of B. Thus $|V| = n + m + 2 = O(n+m)$. For each element $B[i,j]$ in B we add the directed edge $r_i \rightarrow c_j$ with capacity $\lceil B[i,j] \rceil$, which is $1 > B[i,j]$ unless $B[i,j] = 0$, in which case the capacity is zero. Finally, we add directed edges $s \rightarrow r_i$ with capacity $w(r_i)$ for each row r_i and add directed edges $c_j \rightarrow t$ with capacity $w(c_j)$ for each column c_j. Note that G has only integral capacities: those out of s and into t are integral by the above case, and the capacities between row and column.

We claim that G has max-flow value at least W if and only if B can be rounded.

[1] Assuming A only has elements between 0 and 1 is helpful in one direction of the proof; if one uses the construction with A instead of B, then it is correct but harder to prove it.

[2] As we later prove in part b, it would be correct at this point to report POSSIBLE, but proving b requires the rest of our reduction, so we proceed. To obtain the rounding of B itself, not only report whether B can be rounded, performing the reduction in full is needed.

Problem 2 continued on next page...

Indeed, if B can be rounded to matrix B', we put $B'[i,j] \leq \lceil A[i,j] \rceil$ flow on edge $r_i \to c_j$ for each i, j. The sums of columns and sums of rows in B' are the same as those in B, so putting $w(r_i)$ flow on edges $s \to r_i$ and putting $w(c_j)$ flow on edges $c_j \to t$ makes the resulting flow satisfy conservation. Furthermore, the value is clearly W as the edges leaving s are saturated (and the edges into t are saturated) with total capacity W.

Next, suppose G has a max-flow f with value W, which we assume to be integral as G has integral capacities. f saturates all edges out of s and all edges into t. Then the flow in and out of r_i is $w(r_i)$ and the flow in and out of c_j is $w(c_j)$. Next, see that $\lfloor B[i,j] \rfloor = 0$ since all elements of B are less than 1. Since f is feasible, we have $\lfloor B[i,j] \rfloor \leq f(r_i \to c_j) \leq \lceil B[i,j] \rceil$ for all i, j, where the second inequality follows from the capacities of the edges.[3] Furthermore, we have that $f(r_i \to c_j)$ is equal to either $\lceil B[i,j] \rceil$ or $\lfloor B[i,j] \rfloor$ since f is integral and these values are consecutive integers. Thus, the matrix $B'[1..m, 1..n]$ of B where $B'[i,j] = f(r_i \to c_j)$ is a rounding of B.

We conclude the runtime analysis. Constructing B takes $O(n+m)$ time. G can be constructed in $O(E+V) = O(nm)$ time. Since $W \leq mn$, which is the capacity of the edges leaving s (entering t), the max-flow is at most mn. Thus Ford-Fulkerson takes $O(Emn) = O(m^2n^2)$ time. On the other hand, Orlin's algorithm has runtime $O(VE) = O((n+m)mn)$. Thus we use Orlin's algorithm whose runtime dominates the overall runtime: $O((n+m)mn)$.

b. In the previous part, we argued that if A has a sum of columns or sum of rows which is not an integer, there is no way to round A. Thus it remains to prove that if A indeed has its sums of columns as integers and sums of rows as integers, then there is indeed a way to round A. Consider the flow network G and matrix B constructed in the previous part. We consider the (fractional) flow f obtained by pushing $B[i,j]$ flow along the path $s \to r_i \to c_j \to t$. This is feasible and has flow equal to W, the sum of all elements in B. Thus the max-flow in B has value at least W. Furthermore, the max-flow value is at most B (as it is the sum of capacities out of s and the sum of capacities into t). So f is a max-flow. As argued above, there is a rounding of B if the max-flow is W, so B (and thus A) can be rounded.

[3] This is where it is easier to work with B instead of A: it is harder to justify that there is a max-flow with $\lfloor A[i,j] \rfloor \leq f(r_i \to c_j)$.

Problem 3

A *Hamiltonian cycle* is a (simple) cycle that visits all vertices in the graph exactly once. In Monday's lecture, we will see that DIRHAMCYCLE, detecting whether a given directed graph has a Hamiltonian cycle, is NP-hard. The UNDIRHAMCYCLE problem is the same except that the given graph and cycle to detect are undirected.

Describe a polynomial-time reduction from DIRHAMCYCLE to UNDIRHAMCYCLE, and justify its correctness in order to conclude that UNDIRHAMCYCLE is NP-hard. *[Hint: Consider replacing each node u with multiple copies of u. How many copies should be made, how should the copies of one node be connected via edges, and how should the copies of different nodes be connected based on the original edges?*]

Solution: To prove that UNDIRHAMCYCLE is NP-hard, we introduce a polynomial-time reduction from DIRHAMCYCLE to UNDIRHAMCYCLE. If DIRHAMCYCLE is NP-hard, then UNDIRHAMCYCLE is also NP-hard.

Reduction
Let $D = (V_D, E_D)$ be an instance of a graph that contains a DIRHAMCYCLE. We build a new graph $U = (V_U, E_U)$ from D as follows:

- For each $v \in V_D$, we create 2 additional vertices v^- and v^+. Add (v^-, v, v^+) to V_U.
- Connect the new vertices with edges (v^-, v) and (v, v^+). Add these edges to E_U.
- Replace each directed edge $(a, b) \in E_D$ with the edge (a^+, b^-). Add these edges to E_U.

This reduction can be done in time $O(3|V_D| + 2|E_D|) = O(V + E)$.

Claim: There exists an undirected Hamiltonian cycle in U if and only if there exists a directed Hamiltonian cycle in D.

Proof: We will prove both directions of the necessary and sufficient claim above.

$\Longrightarrow$: If there is an undirected Hamiltonian cycle C in U , then there must be a directed Hamiltonian cycle C' in D. This is due to the following.

Each v^- and v^+ in V_U is connected to a vertex v that exists in V_D. Moreover, the construction of the edges E_U ensures that each v in C is immediately preceded by v^- and followed by v^+. Since we added the edge (a^+, b^-) for each directed edge $(a, b) \in E_D$, if v^+ was followed by u^- in C, (v, u) is an edge in E_D. Thus, we can construct a directed hamiltonian cycle C' in D, vertices appearing in the same order as they appear in C.

$\Longleftarrow$: If there is a directed Hamiltonian cycle C' in D , then there must be an undirected Hamiltonian cycle C in U. This is due to the following.

If a directed Hamiltonian cycle exists in D, then it includes all the vertices $v \in V_D$. In U, each v is connected to its corresponding v^- and v^+. Since we added the edge (a^+, b^-) for each directed edge $(a, b) \in E_D$, if v was followed by u in C, then v^+ is connected to u^-. Thus, we can construct an undirected hamiltonian cycle C in U, by simply replacing each $v \in C'$ with v^-, v, v^+, and maintaining the relative ordering.

Problem 4

In this problem we prove that, as for many NP-hard problems, allowing constant additive error does not make the problem any "easier." We define the AlmostMaxIndSet problem as follows: Given an undirected graph $G = (V, E)$, output an integer k in the range $|S^*| - 330 \leq k \leq |S^*|$, where $S^* \subseteq V$ is a maximum independent set in G. Prove that AlmostMaxIndSet is NP-hard. Here are some hints:

- *There is nothing special about the constant 330. Assume it is, say, 3 instead when thinking about the problem.*
- *Reduce from* MaxIndSet *and modify the graph in such a way that the size of the maximum independent set in the resulting graph increases in a controlled, predictable way.*
- *An* **incorrect** *reduction is to simply add many isolated vertices (e.g., ∼330) to the graph. Why is this incorrect? What can you add a sufficiently large amount of instead to obtain a correct reduction?*
- *The size of any independent set is an integer.*

Solution: Reduction:
We will show a polytime reduction from MaxIndSet to AlmostMaxIndSet to show AlmostMaxIndSet is NP-Hard. That is, given a polytime algorithm for AlmostMaxIndSet , we will give a polytime algorithm for MaxIndSet.
Construct a new graph $G' = (V', E')$, where we have 331 copies for each vertex in V, and two copies are joined if and only if the original vertices were joined as well. More formally, we have

$$V' = \{v^{(i)} : i \in [331], v \in V\}, \qquad E' = \{(u^{(i)}, v^{(i)}) : (u, v) \in E, i \in [331]\}$$

In particular, note that we do not have any edge between 2 copies of the same vertex, i.e., $(v^{(i)}, v^{(j)}) \notin E'$.

Our algorithm for MaxIndSet for a graph G is as follows:
Given a graph G, construct G' and return $\lceil \text{AlmostMaxIndSet}(G')/331 \rceil$.

Proof of Correctness: To prove correctness, we need to show that G has a maximum independent set of size k if and only if $\lceil \text{AlmostMaxIndSet}(G')/331 \rceil = k$.

Lemma A: If G has a maximum independent set of size k, then G' has a maximum independent set of size $331k$, and if G' has a maximum independent set of size k, then G has a maximum independent set of size $\lceil k/331 \rceil$. We prove this lemma next.

- If there is an independent set of size k in G, then G' has an independent set of size $331k$. This is because if I is an independent set of size k in G, then

  $$I' = \{v^{(i)} : v \in I, i \in [331]\}$$

  is an independent set in G' of size $331k$, as two copies are joined if and only if the original vertices were joined as well.
- On the other hand, if there is an independent set of size k in G', then G has an independent set of size $\lceil k/331 \rceil$.

Problem 4 continued on next page...

This is because G' consists of 331 copies of G, and so G' is the union of 331 (potentially unequal) independent sets in G. Since $|I| = k$, at least one of the copies of G in G' has an independent set of size at least the average, $k/331$. Furthermore, the size of any independent set is integral, so there is an index $i \in [331]$ such that the independent set

$$I = \{v : v^{(i)} \in I'\}$$

in G has size at least $\lceil k/331 \rceil$.

- Thus, if G has a maximum independent set of size k, G' has an independent set of size $331k$, and this must be a maximum independent set as otherwise G would have an independent set of size at least $\lceil \frac{331k+1}{331} \rceil > k$. Similarly, if G' has a maximum independent set of size k, then G has an independent set of size $\lceil k/331 \rceil$, and this must be a maximum independent set as otherwise G' would have an independent set of size $> 331\lceil k/331 \rceil \geq k$.

We are now ready to show that this is a valid polytime reduction.

- Firstly, the reduction is clearly polytime as the number of vertices in G' is $331|V| = O(V)$ and the number of edges is at most $331^2|V|^2 = O(V^2)$, and moreover G' can be constructed in $O(V + E) = O(V^2)$ time as well.
- We will now prove the first direction of our reduction. (G has maximum independent set of size $k \implies 331k - 330 \leq \textsc{AlmostMaxIndSet}(G') \leq 331k$)

  By Lemma A, we know that the maximum independent set in G' has size $331k$, thus by definition $331k - 330 \leq \textsc{AlmostMaxIndSet}(G') \leq 331k$.
- We will now prove the second direction of our reduction. ($331k - 330 \leq \textsc{AlmostMaxIndSet}(G') \leq 331k \implies G$ has maximum independent set of size k)

  Note that by definition we have that $|S^*| \geq \textsc{AlmostMaxIndSet}(G') \geq |S^*| - 330$, where S^* is a maximum independent set in G'. By Lemma A, we know that $|S^*| = 331\alpha$, where α is the size of a maximum independent set in G. We must have $\alpha = k$ because

  $$\alpha > k \implies \textsc{AlmostMaxIndSet}(G') \geq 331(k+1) - 330 > 331k, \text{ contradiction.}$$
  $$\alpha < k \implies \textsc{AlmostMaxIndSet}(G') \leq 331(k-1) < 331k - 330, \text{ contradiction.}$$